EXPERIMENT 2

Implement Divide and Conquer Based Merge Sort and Quick Sort and Compare Their Performance for the Same Set of Elements

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement Merge Sort and Quick Sort using the divide and conquer technique in C and compare their performance for the same set of elements.

2. Objectives

• Implement Merge Sort using the divide and conquer approach.

• Implement Quick Sort using the divide and conquer approach.

• Sort the same input elements using both algorithms.

• Measure and compare the execution times of both algorithms.

• Analyze their time and auxiliary-space complexities.

3. Theory

Divide and conquer is an algorithm design technique that divides a problem into smaller subproblems, solves them recursively, and combines their results when required.

Merge Sort: It divides the array into two halves, recursively sorts each half, and merges the sorted halves into one sorted array.

Quick Sort: It selects a pivot element, partitions the array around the pivot, and recursively sorts the subarrays on either side of the pivot.

4. Algorithm

A. Merge Sort

1. If the subarray contains one or zero elements, return.

2. Find the middle index and divide the array into left and right halves.

3. Recursively sort the left half.

4. Recursively sort the right half.

5. Merge the two sorted halves into a single sorted array.

B. Quick Sort

1. If the subarray contains one or zero elements, return.

2. Choose the last element as the pivot.

3. Partition the array so elements less than or equal to the pivot come before it and larger elements come after it.

4. Place the pivot in its correct sorted position.

5. Recursively sort the left and right partitions.

5. Sample Input and Output

Sample input: 6 elements — 38, 12, 27, 9, 43, 18.

Both algorithms produced the sorted sequence: 9, 12, 18, 27, 38, 43.

In the recorded run, both execution times displayed as 0.00000000 seconds. This is because the input was very small and the clock timer did not register a measurable interval; it does not mean the algorithms take literally zero time.

6. Complexity Analysis

Case / Parameter	Merge Sort	Quick Sort
Best-case time	O(n log n)	O(n log n)
Average-case time	O(n log n)	O(n log n)
Worst-case time	O(n log n)	$O(n^2)$
Auxiliary space	O(n)	Average O(log n) stack

7. Performance Comparison

Both algorithms successfully sorted the same input. Merge Sort guarantees O(n log n) time in the best, average, and worst cases, but needs additional memory for merging. Quick Sort often performs efficiently in practice and generally uses less auxiliary memory, but its worst-case time is $O(n^2)$ with an unfavorable pivot pattern. The measured time for six elements is too small for a meaningful performance conclusion.

8. Result

Merge Sort and Quick Sort were implemented using the divide and conquer technique in C. Both algorithms successfully sorted the same six input elements into the sequence 9, 12, 18, 27, 38, 43. Their theoretical time and space complexities were compared.

9. Screenshots / Evidence

The following screenshots document the source code and successful execution in Visual Studio Code.

Screenshot 1



```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100000

// Merge two sorted subarrays
Windsurf: Refactor | Explain | ×
void merge(int arr[], int temp[], int left,
           int mid, int right) {
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];

    for (i = left; i <= right; i++)
        arr[i] = temp[i];
}

// Merge Sort
Windsurf: Refactor | Explain | ×
void mergeSort(int arr[], int temp[],
               int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, temp, left, mid);
        mergeSort(arr, temp, mid + 1, right);
```

Screenshot 1: Merge function and beginning of the Merge Sort implementation.

Screenshot 2

```
        mergeSort(arr, temp, mid + 1, right);
        merge(arr, temp, left, mid, right);
    }
}

// Partition function for Quick Sort
Windsurf: Refactor | Explain | ×
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }

    int t = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = t;

    return i + 1;
}

// Quick Sort
Windsurf: Refactor | Explain | ×
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
```

Screenshot 2: Completion of Merge Sort and Quick Sort partition and sorting functions.

Screenshot 3



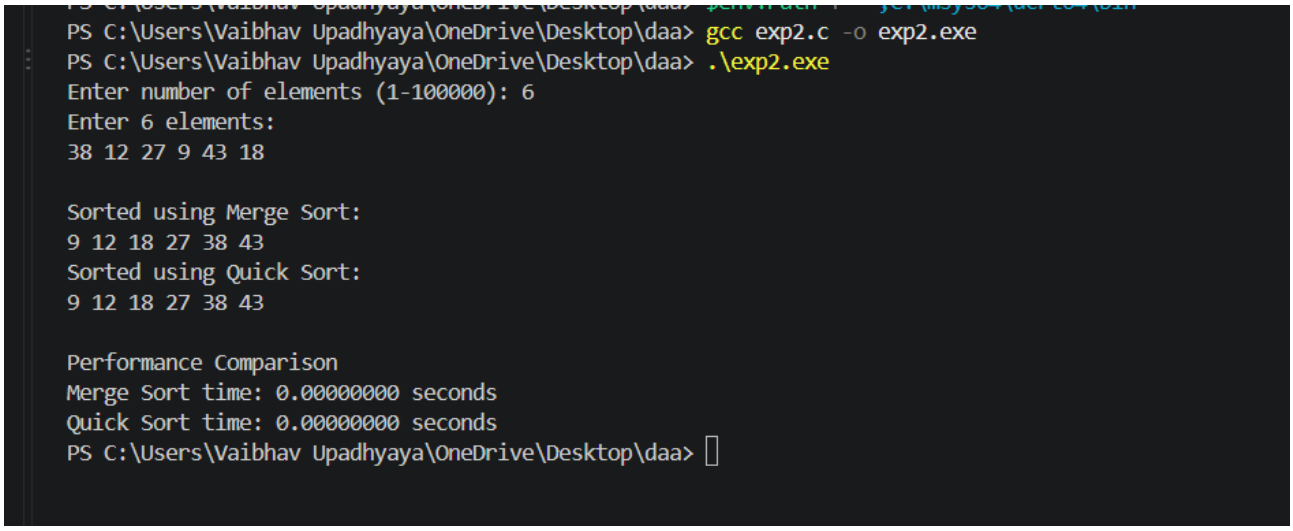
Screenshot 3: Array display function and main() input validation and array setup.

Screenshot 4

```
106     // Copy the same input for both algorithms
107     for (int i = 0; i < n; i++) {
108         mergeArr[i] = original[i];
109         quickArr[i] = original[i];
110     }
111
112     // Measure Merge Sort
113     start = clock();
114     mergeSort(mergeArr, temp, 0, n - 1);
115     end = clock();
116     mergeTime = (double)(end - start) / CLOCKS_PER_SEC;
117
118     // Measure Quick Sort
119     start = clock();
120     quickSort(quickArr, 0, n - 1);
121     end = clock();
122     quickTime = (double)(end - start) / CLOCKS_PER_SEC;
123
124     printf("\nSorted using Merge Sort:\n");
125     display(mergeArr, n);
126
127     printf("Sorted using Quick Sort:\n");
128     display(quickArr, n);
129
130     printf("\nPerformance Comparison\n");
131     printf("Merge Sort time: %.8f seconds\n", mergeTime);
132     printf("Quick Sort time: %.8f seconds\n", quickTime);
133
134     return 0;
135 }
136
```

Screenshot 4: Copying the same input for both algorithms and measuring execution times.

Screenshot 5



Screenshot 5: Successful program execution showing matching sorted output and measured times.